

Interfaces, Packages and Multithreaded Programming

Interfaces: An interface in Java is a blueprint of a class. It has static constants and abstract methods. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java. The interface keyword is used to declare an interface.

Writing an interface is similar to writing a class in following ways

- An interface can contain any number of methods.
- An interface is written in a file with a .java extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in a .class file.
- Interfaces appear in packages, and their corresponding bytecode file must be in a directory structure that matches the package name.

an interface is different from a class in several ways, including –

- You cannot instantiate an interface. ○ An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

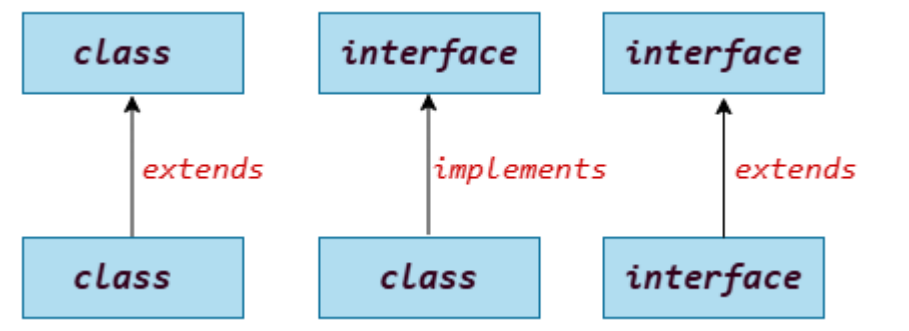
Syntax :

```
interface <interface_name>
{
    // declare constant fields
    // declare methods that abstract by default.
}
```

Ex:

```
interface Ex
{
    final int a = 10; // public, static and final
    void display(); // public and abstract
}
```

Relationship Between Classes and Interfaces:



Extending Interfaces: An interface can extend another interface in the same way that a class can extend another class. The extends keyword is used to extend an interface, and the child interface inherits the methods of the parent interface.

Syntax:

```
interface name2 extends name1
{
    Body of name2
}
```

EXAMPLE:

```
interface Printable
{
    void print();
}
interface Showable extends Printable
{
    void show();
}
```

Implementing Interfaces: User Defined Interfaces can be inherited in class using 'implements' keyword. More than one interfaces can be implemented separated by comma in a single class.

```
Class classname implements interfacename
{
    Body of the classname
}
```

Ex:

```
interface Printable
{
    public void print();
}
class Example implements Printable
{
    public void print()
    {
        System.out.println("printing");
    }
    public static void main(String args[])
    {
        Example e1=new Example();
        e1.print();
    }
}
```

Multiple inheritance is not supported in the case of class because of ambiguity. This ambiguity is resolved by implementing interfaces.

```
interface Printable
{
public void print();
}
Interface Showable
{
Public void show();
}
class Example implements Printable,Showable
{
public void print()
{
System.out.println("printing");
}
public void show()
{
System.out.println("showing");
} public static void main(String args[])
{
Example e1=new Example();
e1.print();
e1.show();
}
}
```

- As java does not support multiple inheritance due to ambiguity in calling same method in more than one classes and it can be achieved by implementing interfaces,since interfaces will consists only abstract methods.
- In the given example class 'Example' is implementing multiple interfaces and also defining methods show() and print() completely,thereby illustrating multiple inheritance.

Accessing Interface Variables: Copy of the fields of an interface are available in the class that implements it.Since all the fields of an interface are static by default, we can access them using the name of the interface as

```
interface A
{
int x=100;
public void show();
}

interface B extends A
{
public void view();
}
```

```
public class Demo implements B
{
    int x=150;
    public void show()
    {
        System.out.println("show() from interface A");
    }

    public void view()
    {
        System.out.println("view() from interface B");
    }

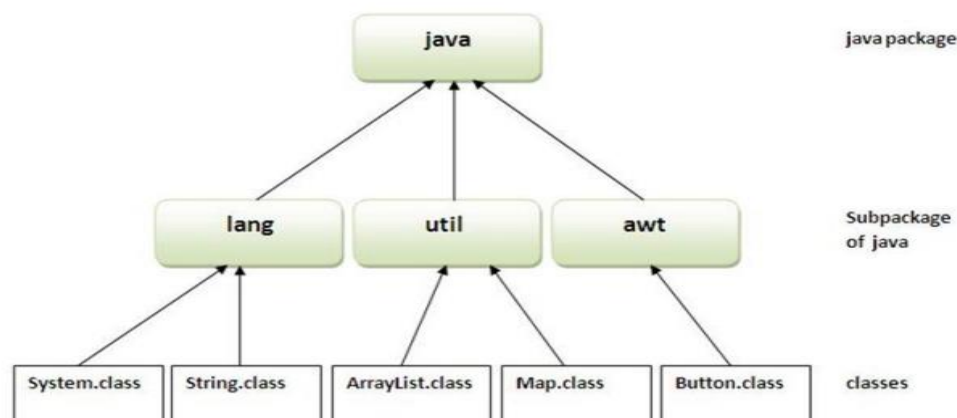
    public static void main(String args[])
    {
        Demo d1=new Demo();
        d1.show();
        d1.view();
        System.out.println("x value from class
        Demo:"+d1.x); System.out.println("x value from
        interface A:"+A.x);
    }
}
```

Packages: A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc. Using package it becomes easier to locate the related classes. Packages are used to control access, to make searching/locating and usage of classes, interfaces, enumerations and annotations easier, etc.

Java API Packages:

These packages consist of a large number of classes which are a part of Java API. Some of the commonly used built-in packages are:

- 1) java.lang: Contains language support classes(e.g classed which defines primitive data types, math operations). This package is automatically imported.
- 2) java.io: Contains classed for supporting input / output operations.
- 3) java.util: Contains utility classes which implement data structures like Linked List, Dictionary and support ; for Date / Time operations.
- 4) java.applet: Contains classes for creating Applets.
- 5) java.awt: Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc).
- 6) java.net: Contain classes for supporting networking operations.



Using System Packages: The packages are organised in a hierarchical structure. Package named java contains the package awt which in turn contains various classes required for implementing graphical user interface. Hierarchy is represented by separating levels with dots.

import packagename.classname;

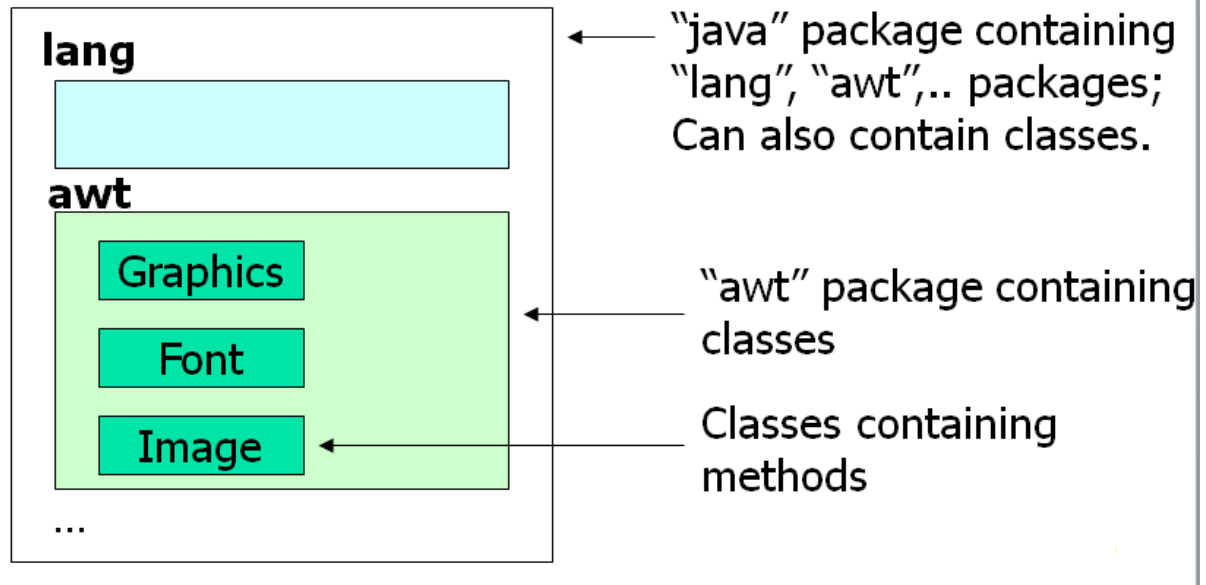
or

import package.*

Ex; import java.awt.Color;

import java.awt.*

java



Naming Conventions: Packages can be named using the standard Java naming rules. Packages begin with lowercase letters. Class names begin with uppercase letters. This helps to distinguish between package names and class names. Methods begin with lowercase letters.

```
double y= java.lang.Math.sqrt(x);
```

Creating Packages: Creating packages in Java involves organizing related classes and interfaces into a single unit. They help organize your code and avoid class name conflicts.

Creating our own package involves the following steps:

- Declare the package at the beginning of a file using the form
- Define the class that is to be put in the package and declare it public.
- Create a subdirectory under the directory where the main source files are stored.
- Store the listing as the classname.java file in the subdirectory created.
- Compile the file. This creates .class file in the subdirectory.

```
package mypackage;
public class MyClass
{
    public void display()
    {
        System.out.println("This is MyClass in mypackage.");
    }
}
```

Accessing a Package: There are three ways to access the package from outside the package.

- 1.import package.*;
- 2.import package.classname;
- 3.fully qualified name.

1. Using `packagename.*` :

- If you use `package.*` then all the classes and interfaces of this package will be accessible but not subpackages.
- The `import` keyword is used to make the classes and interface of another package accessible to the current package.

//save by A.java

```
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}
```

//save by B.java

```
package mypack;
import pack.*;
class B
{
    public static void main(String args[])
    {
        A obj = new A(); obj.msg();
    }
}
```

Output: Hello

2. Using `packagename.classname`:

- If you import `package.classname` then only declared class of this package will be accessible.

//save by A.java

```
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}
```

//save by B.java

```
package mypack;

import pack.A;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.msg();
    } }
```

Output: Hello

3. Using fully qualified name:

- If you use fully qualified name then only declared class of this package will be accessible.
- Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.
- It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

//save by A.java

```
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}
```

//save by B.java

```
package mypack;
class B
{
    public static void main(String args[])
    {
        pack.A obj = new pack.A(); //using fully qualified name
        obj.msg();
    }
}
```

Using A Package:

Demo1.java

Package P1;

```
public class Demo1
{
    public void show()
    {
        System.out.println("show() from class Demo1 from package P1");
    }
}
```

Demo2.java

package P2;

import P1.*;

public class Demo2

```
{
    public void view()
    {
        System.out.println("view() from class Demo2 from package P2");
    }
    public static void main(String args[])
    {
        Demo1 d1=new Demo1();
        Demo2 d2=new Demo2();
        d1.show(); d2.view(); }
}
```

Adding A Class to a Package:

- If you want to put two public classes in a package, have two java source files containing one public class, but keep the package name same.

- For example:

```
//save as A.java
package BCA;
public class A{}
```

```
//save as B.java
package BCA;
public class B{}
```

Hiding Classes: when we import a package using * all public classes are imported. If certain classes should not be imported then we hide the classes from accessing outside of the package.

```
package pack1;
public class C1
{
    public void print()
    {
        System.out.println("Printing from class C1 of package1");
    }
}
class B1
{
    public void show()
    {
        System.out.println("Printing from class B1 of package1");
    }
}
```

```
package pack2;
import pack1.*;
class S1
{
    public static void main(String args[])
    {
        C1 c=new C1();
        c.print();
        B1 b=new B1(); //ERROR SINCE CLASS B IS NOT PUBLIC
        b.show();
    }
}
```

Multithreading Programming:

A Thread may be defined as a separate sequential flow of control within the main program. It is a smallest unit of execution within a process. Threads are used in a program for achieving specific tasks. Multithreading means, using different threads to perform different parts of a task. In Multithreading, a task is divided into different subtasks and different threads, which run simultaneously, for executing these subtasks. Since threads are subprograms of main application program and share the common memory space, they are known as lightweight threads or lightweight processes.

Creating Threads: Java defines two ways by which a thread can be created.

1. By implementing the Runnable interface.
2. By extending the Thread class

By Implementing the Runnable Interface: Threads are implemented in the form of objects that contain a method called run(). The run() method is the heart of any thread. It makes up the entire body of the thread and is the only method in which the threads behaviour can be implemented.

EX:

```
class A implements Runnable
{
    public void run()
    {
        System.out.println("thread started running..");
    }
}
Class Demo
{
    public static void main( String args[] )
    {
        A obj = new A();
        Thread t = new Thread(obj); //passing class instance in Thread object's constructor
        t.start();
    }
}
```

Output : thread started running..

Note:

- ☉ To call the run() method, start() method is used.
- ☉ On calling start(), a new stack is provided to the thread.

By Extending the Thread Class: This is another way to create a thread by a new class that extends Thread class.

EX:

```
class A extends Thread
{
    public void run()          // implementing the run() method
    {
        System.out.println("thread started running..");
    }
}
Class MyThreadDemo{ public static void main( String args[] )
{
    A obj = new A();
    obj.start();
}
```

```
}}      Output : thread started running..
```

Stopping and Blocking A Thread:

Stopping: To stop a thread from running further we use stop() method. This statement causes the thread to move to the dead state.

```
aThread.stop();
```

Blocking: Blocking thread is nothing but suspending the execution of thread temporarily. Threads can be blocked using different methods such as:

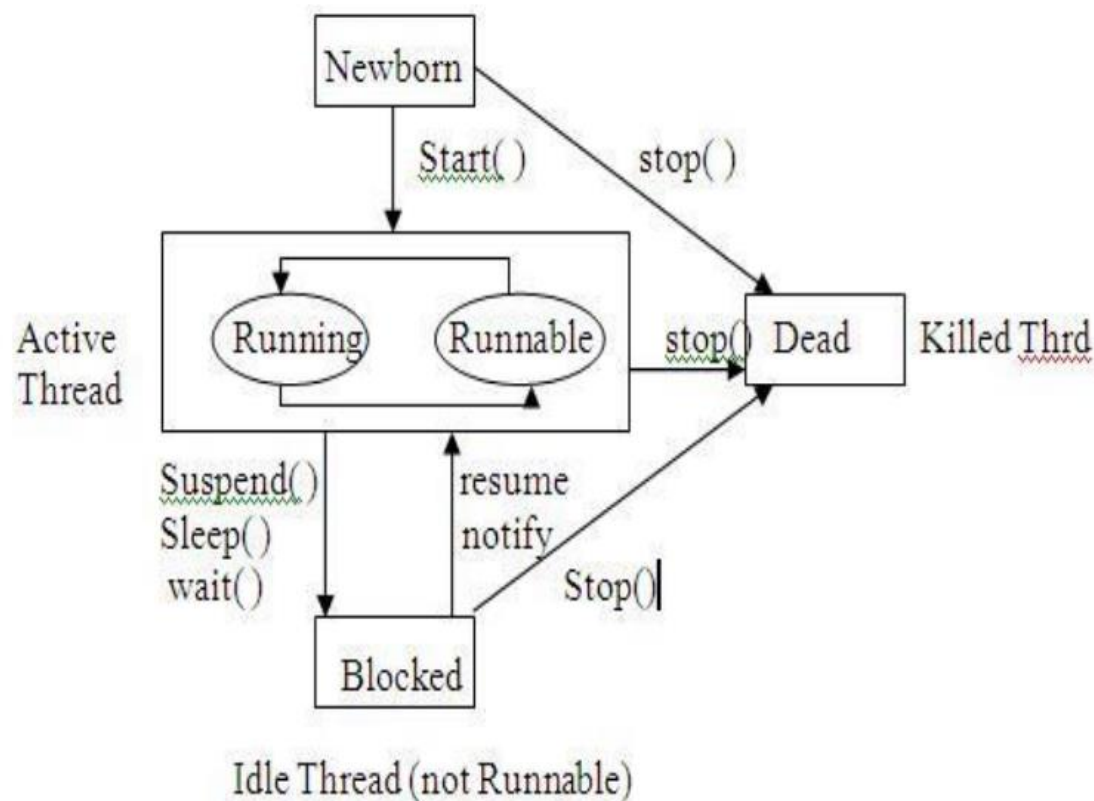
- ☐ Sleep():blocks thread for specified time
- ☐ Suspend():blocks thread until next order
- ☐ Wait():blocks thread until certain condition occurs.

```
public class Multithread extends Thread
{
    public void run()
    {
        try
        {
            // Displaying the thread that is running
            System.out.println( "Thread " + Thread.currentThread().getName() + " is running");
        }
        catch (Exception e)
        {
            // Throwing an exception
            System.out.println("Exception is caught");
        }
    }
}

public class MultithreadDemo
{
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++)
        {
            Multithread m1= new Multithread();
            m1.start();
        }
    }
}
```

Life Cycle of a Thread: during life cycle there are many states. They include:

1. Newborn state
2. Runnable state
3. Running state
4. Blocked state
5. Dead state



1. New born State:

- When we create a thread object, the thread is born and is said to be in newborn state.
- The thread is not yet scheduled for running.
- At this state, we can do only one of the following things with it:
 - * Schedule it for running using start() method.
 - * Kill it using stop () method.

2. Runnable State:

- The Runnable state means that the thread is ready for execution and is waiting for the availability of the processor.
- That is, the thread has joined the queue of threads that are waiting for execution.

3. Running State:

- Running means that the processor has given its time to the thread for its execution.
- The thread runs until it relinquishes (give up) control on its own.
- It has been suspended using suspend() method.
- A suspended thread can be revived by using the resume() method.
- we can put a thread to sleep for a specified time period using the method sleep (time) where time is in milliseconds.
- This means that the thread is out of the queue during this time period.
- It has been told to wait until some events occur.
- This is done using the wait () method.
- The thread can be scheduled to run again using the notify () method.

4. Blocked State:

- A thread is set to be blocked when it is prevented from entering into the Runnable state and subsequently the running state.
- This happens when the thread is suspended, or sleeping, or waiting in order to satisfy certain requirements.

5. Dead State:

- Every thread has a life cycle.
- A running thread ends its life when it has completed executing its run () method.
- It is a natural death.
- However, we can kill it by sending the stop message to it at any state thus causing a premature death to it.
- A thread can be killed as soon it is born, or while it is running, or even when it is in “not
- Runnable” (blocked) condition.

THREAD CLASS METHODS Method Description

start() It is used to start the execution of the thread.

run() It is used to do an action for a thread.

sleep() It sleeps a thread for the specified amount of time.

currentThread() It returns a reference to the currently executing thread object.

setPriority() It changes the priority of the thread.

getPriority() It returns the priority of the thread.

isAlive() It tests if the thread is alive.

getId() It returns the id of the thread. Method Description

suspend() It is used to suspend the thread.

resume() It is used to resume the suspended thread.

stop() It is used to stop the thread.

destroy() It is used to destroy the thread group and all of its subgroups.

interrupt() It interrupts the thread.

notify() It is used to give the notification for only one thread which is waiting for a particular object. notifyAll() It is used to give the notification to all waiting threads of a particular object.

getState() It returns the state of the thread.

getName() It returns the name of the thread.

SYNCHRONIZATION

When multiple threads are competing for same resources then they may lead to serious problems. For Example, if thread t1 is trying to read a record from file while another thread t2 is writing the same file then thread t1 may read wrong data. This problem can be eliminated by using synchronization technique in java.

Synchronization in java is the capability to control the access of multiple threads to any shared resource.

```
class Table
{
synchronized void printTable(int n)
{
for(int i=1;i<=5;i++)
{
System.out.println(n*i);
Try
```

```

{
Thread.sleep(400);
}
catch(Exception e)
{ System.out.println(e);
}
}
}
}
class MyThread1 extends Thread
{
Table t;
MyThread1(Table t)
{
this.t=t;
}
public void run()
{
t.printTable(5); } }
class MyThread2 extends Thread
{
Table t;
MyThread2(Table t)
{
this.t=t;
}
public void run()
{
t.printTable(100);
}
}
class TestSynchronization1
{
public static void main(String args[])
{
Table obj = new Table();
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}

```